

CS6493 NLP Closed-Book Review - Lectures 1-12

这份复习笔记按 lecture 顺序整理。每个知识点都给出 English + 中文解释，适合闭卷考试前快速背诵、理解和默写。

How to Use This File / 这份文件怎么用

你说得对，闭卷考试最重要的不是“知道 PPT 讲过什么”，而是看到题目以后能立刻判断：

1. 这题在考哪个 lecture 的哪个考点？
2. 这是定义题、比较题、设计题还是计算题？
3. 我应该按什么步骤写，才能拿到过程分？

所以从这里开始，先给你一份 **exam-oriented map / 考试导向地图**。后面按 lecture 的内容可以当作详细解释查阅。

Chinese-First Exam Summary / 中文优先考前总览

如果你现在只剩很少时间，请先背这一节。它把 PPT、quiz、homework 里最可能考的内容用中文讲清楚。

1. 数据预处理与分词

NLP 的第一步通常是把原始文本变成模型能处理的 token。token 可以是词、字符或子词。词级分词好理解，但容易遇到 OOV；字符级分词几乎没有 OOV，但序列会变得很长；子词分词是折中方案，BPE 就是典型子词分词方法。

考试最可能问：

- tokenization 是什么？
- word / character / subword tokenization 的优缺点是什么？
- BPE 怎么一步步 merge？
- 为什么 BPE 能缓解 OOV？

答题重点：

BPE 从字符开始，每次统计当前语料中最频繁的相邻符号对，并把它合并成新 token。合并后要重新统计 pair。最终遇到新词时，可以用已学到的子词拆分，所以比 word-level 更不容易 OOV。

2. 语言模型与 n-gram

语言模型的本质是给一个词序列赋予概率，也可以理解为根据前文预测下一个词。

$$[P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_1, \dots, w_{t-1})]$$

n-gram 用 Markov 假设简化这个问题:

$$[P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-n+1}, \dots, w_{t-1})]$$

考试最可能问:

- bigram/trigram 怎么列?
- n-gram 为什么有稀疏问题?
- Markov assumption 有什么限制?
- perplexity 是什么?

答题重点:

n-gram 只看前 (n-1) 个词, 因此不能建模很长距离依赖。可能的 n-gram 数量随 (V^n) 增长, 所以需要很大语料, 否则很多合理组合没见过, 会出现 0 概率或不可靠估计。

2.5 交叉熵、负对数似然与困惑度

交叉熵是分类和语言模型里非常重要的 loss。它衡量真实分布和模型预测分布之间的差距。真实标签通常是 one-hot, 例如正确类别是第 (k) 类, 那么:

$$[y_k=1, \text{quad } y_i=0 \ (i \neq k)]$$

交叉熵:

$$[L = -\sum_{i=1}^C y_i \log \hat{y}_i]$$

因为只有正确类别的 ($y_k=1$), 所以化简为:

$$[L = -\log \hat{y}_k]$$

中文直觉:

模型给正确答案的概率越高, loss 越小; 模型给正确答案的概率越低, loss 越大。

例子:

如果真实类别是 class 2, 模型给 class 2 的概率是 0.8:

$$[L = -\log(0.8) \approx 0.223]$$

如果模型只给正确类别 0.1:

$$[L = -\log(0.1) \approx 2.303]$$

语言模型中, 每一步预测下一个词, 本质上是在词表上做多分类。如果真实下一个词是 (w_t), 模型给它的概率是 ($P(w_t | w_{1:t-1})$) 一句话的平均交叉熵:

$$[L = -\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{1:t-1})] \text{ 困惑度 perplexity 和交叉熵的关系:}$$

$$[PP = \exp(L)]$$

所以:

- 交叉熵越小, 困惑度越小。
- 困惑度越小, 说明模型越不“困惑”, 越能预测真实文本。

考试最可能问：

- cross entropy 是什么？
- 给预测概率和真实标签，怎么算 cross entropy？
- cross entropy 和 perplexity 有什么关系？
- 为什么 language model 用 cross entropy 训练？

3. one-hot 与词向量

one-hot 把每个词表示成一个只有一个位置为 1、其余为 0 的向量。它的问题是所有不同词之间都正交，内积为 0，因此无法表达语义相似度。

词向量是低维稠密向量，语义相近的词在向量空间里更接近。它基于分布式假设：出现在相似上下文中的词通常有相似含义。

考试最可能问：

- one-hot 怎么写？
- 两个不同 one-hot 的 dot product 是多少？
- embedding table 参数量怎么算？
- word2vec 和 GloVe 有什么区别？
- ELMo 解决了什么问题？

答题重点：

embedding table 参数量 = 词表大小 $\times$ 嵌入维度。word2vec 通过局部上下文预测学习词向量，GloVe 通过全局共现矩阵学习词向量。ELMo/BERT 这类上下文化表示能根据句子上下文给同一个词不同向量，因此能缓解一词多义问题。

4. word2vec、负采样与 GloVe

CBOW 是用上下文预测中心词；Skip-gram 是用中心词预测上下文词。Skip-gram 如果使用 full softmax，每次都要遍历整个词表，复杂度是 $O(|V|)$ ，所以大语料上很慢。

负采样只更新一个正样本和 (K) 个负样本，把复杂度降到 $O(K)$ 。Hierarchical softmax 把词表组织成树，把复杂度降到 $O(\log |V|)$ 。

GloVe 的思想是让词向量内积接近共现次数的对数：

$$[w_i^T \tilde{w}_j + b_i + \tilde{b}_j] \approx \log X_{ij}$$

考试最可能问：

- skip-gram 为什么慢？
- negative sampling 为什么快？
- GloVe 和 word2vec 的核心区别？

5. Transformer 与 Attention

Attention 的核心是 query、key、value。query 表示“我要找什么”，key 表示“每个位置有什么可匹配的特征”，value 表示“真正要取出的内容”。

$$[\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V]$$

考试最可能问：

- attention 公式是什么？
- 为什么要除以 $(\sqrt{d_k})$ ？
- causal mask 为什么要在 softmax 前加？
- 4x4 mask 怎么写？
- 如果 $(W^Q=W^K)$ 会发生什么？

答题重点：

mask 要在 softmax 前加，因为未来位置要变成 $(-\infty)$ ，这样 softmax 后概率才是 0，并且不会影响分母。如果 $(W^Q=W^K)$ ，score matrix $(S=QK^T=ZZ^T)$ 会对称，限制模型表达有方向的 attention。

6. BERT、GPT 与预训练微调

BERT 是 encoder-only，使用双向 attention，适合理解任务，如分类、NER、抽取式 QA。GPT 是 decoder-only，使用 causal masked attention，适合生成任务，如续写、对话、摘要生成。

预训练是在大规模文本上学习通用语言知识，微调是在具体任务标注数据上适配模型。

考试最可能问：

- BERT 和 GPT 的区别？
- 为什么 BERT 适合理解，GPT 适合生成？
- pretraining 和 fine-tuning 区别？

7. NLU、NLG 与任务分类

NLU 是理解文本，例如情感分析、文本分类、意图识别、NER、阅读理解。NLG 是生成文本，例如机器翻译、摘要、对话生成。

分类题常见类型：

- binary classification: 二分类，例如 spam vs legitimate。
- single-label multi-class: 多分类但只选一个，例如新闻类别。
- multi-label classification: 一个样本可有多个标签，例如论文主题。
- ordinal classification: 类别有顺序，例如评分等级。

考试最可能问：

- 某任务属于 NLU 还是 NLG？
- 某分类场景属于哪种分类？
- precision/recall/F1 怎么解释？

8. QA 与 SQuAD

抽取式 QA 的答案是从 passage 中直接截取 span。模型通常预测 start index 和 end index。SQuAD 1.1 的答案都在文章中；SQuAD 2.0 加入无答案问题，因此模型还要判断是否应该拒答。

考试最可能问：

- 什么是 extractive QA？
- SQuAD 为什么是强 benchmark？
- SQuAD 2.0 比 1.1 难在哪里？
- EM 和 F1 是什么？

9. 机器翻译、解码与 BLEU

机器翻译是建模：

$P(y|x) = \prod_t P(y_t | y_{<t})$ greedy search 每一步选当前概率最高的词，速度快但可能错过全局最优。beam search 每一步保留 top (k) 个候选序列，更可能找到高概率整体序列，但计算更贵。

BLEU 用 n-gram precision 和 brevity penalty 评估生成结果：

$BLEU = BP \cdot \exp\left(\sum_n w_n \log P_n\right)$

考试最可能问：

- greedy 生成路径概率怎么算？
- beam search (k=2) 怎么展开？
- BLEU 怎么代入公式？
- brevity penalty 惩罚什么？

10. 任务型对话系统

任务型对话系统的目标是帮用户完成具体任务，例如 COVID 结果申报、订票、预约、客服。

核心流程：

$[NLU \rightarrow Dialogue\ State \rightarrow Tracking \rightarrow Dialogue\ Policy \rightarrow Backend/API \rightarrow NLG]$

CRB 题里：

- domain: COVID-19 reporting
- intent: report positive result / report negative result / update report
- slot: name, HKID, test date, test result, phone number

考试最可能问：

- domain、intent、slot 分别是什么？
- NLU 后面还有哪些模块？
- 如何评价 CRB 系统？

11. LLM、Prompt、Alignment、Agent

LLM 是大规模 Transformer 语言模型。Prompting 是通过输入指令、上下文和示例来让模型完成任务。CoT 让模型写出中间推理步骤，常用于数学和逻辑题。Alignment 是让模型更符合人类意图和偏好，RLHF 和 DPO 都是常见对齐方法。

Agent 是由 LLM 驱动、能规划、调用工具、使用记忆并根据反馈完成任务的系统。

考试最可能问：

- prompt 是什么？
- CoT 为什么有用？
- RLHF 大致流程？
- agent 有哪些组成部分？

12. 高效训练、LoRA 与 RAG

混合精度 Adam 训练时，显存不只是模型权重，还包括梯度、fp32 参数副本、momentum、variance，总体大约：

[16Ψ]

LoRA 冻结原始权重，只训练低秩更新：

[$\Delta W = BA$]

RAG 是先检索外部知识，再让 LLM 基于检索证据生成答案。它适合知识更新快、领域性强、需要引用的任务。

考试最可能问：

- 为什么训练 LLM 很占显存？
 - LoRA 为什么参数高效？
 - RAG 为什么能减少幻觉？
 - RAG pipeline 怎么设计？
-

A. High-Priority Exam Points / 高频考点总表

Priority	Topic	Why Important	Likely Question Type	Must Know
Very high	Tokenization and BPE	HW1 directly tested BPE; lecture review mentions OOV/BPE	calculation + explanation	word/char/subword differences, BPE merge steps
Very high	n-gram LM	HW1 tested n-gram extraction and sparsity	calculation + short answer	Markov assumption, ($O(V^n)$), context extraction
Very high	One-hot vs embeddings	HW1 tested one-hot vectors, dot product, parameter count	calculation + explanation	orthogonality, semantic similarity failure
Very high	word2vec / GloVe	HW1 coding and conceptual comparison	explanation + algorithm	CBOW, skip-gram, negative sampling, co-occurrence
Very high	Greedy / Beam / BLEU	HW2 directly tested all three	calculation	path probability, beam width, BLEU formula
Very high	Attention and mask	HW2 directly tested attention constraints and causal mask	derivation + matrix writing	(QK^T), softmax, mask before softmax
High	NLU/NLG and classification types	HW2 and quiz both test task taxonomy	classification + examples	binary, multi-class, multi-label, ordinal
High	QA and SQuAD	HW2 tested extractive QA and SQuAD 2.0	short answer	span prediction, EM/F1, unanswerable questions
High	Task-oriented dialogue	Quiz directly tested CRB chatbot	design question	domain, intent, slot, DST, policy, NLG
High	RAG	HW2 campus assistant design; lecture 11	system design	retrieval, embeddings, vector DB, grounding
Medium-high	BERT vs GPT / Transformer	central lecture topic	comparison	encoder vs decoder, bidirectional vs causal
Medium-high	LLM alignment/prompting	lecture 7-8	definition/comparison	prompt, CoT, RLHF, DPO

Priority	Topic	Why Important	Likely Question Type	Must Know
Medium	PEFT / LoRA / ZeRO	lecture 10	explanation + simple memory	(16\Ps), Adapter vs LoRA

A2. Lecture Exam Question Bank with Steps / Lecture 题型例题与解题步骤

这一节是给你做题用的。每个 lecture 考点都按 题型 -> 例题 -> 解题步骤 -> 参考答案 来写，而且中文解释会更多。闭卷考试时，你可以把这里当作答题模板来套。

Type A: Tokenization and BPE / 分词与 BPE 题

Likely Question / 可能题目

Explain the difference among word tokenization, character tokenization, and subword tokenization. Why can BPE reduce the OOV problem? Given a small corpus and a target vocabulary size, perform BPE merges and tokenize new words.

中文题目：

解释词级分词、字符级分词和子词分词的区别。为什么 BPE 可以缓解 OOV 问题？给定一个小语料和目标词表大小，手动执行 BPE 合并，并对新词分词。

How to Solve / 解题步骤

1. 先定义三种 tokenization。
2. 对比优缺点: word-level 容易 OOV；character-level 序列长；subword-level 折中。
3. 解释 BPE: 从字符开始，反复合并最高频相邻 pair。
4. 如果是计算题，先算需要 merge 几次:

[number\ of\ merges=target\ vocab\ size-base\ vocab\ sized]

1. 每轮统计当前切分下的相邻符号对，选最高频 pair 合并。
2. 对新词分词时，从左到右做 longest match；遇到基础词表没有的字符写 [unk]。

Example / 例题

Base vocabulary:

[o,l,d,e,r,s,t,h,u,g,p]

Words:

[(old,10),(older,5),(oldest,8),(hug,8),(pug,4),(hugs,5)]

Target vocab size = 16. How many merges?

Answer / 参考答案

基础词表大小是 11，目标词表大小是 16，所以：

[$16-11=5$]

需要 5 次 merge。

第一轮统计相邻 pair:

- $((o,l)=10+5+8=23)$
- $((l,d)=10+5+8=23)$
- $((u,g)=8+4+5=17)$
- $((h,u)=8+5=13)$
- $((d,e)=5+8=13)$

题目如果指定第一步合并 `o+l`，就合并: [`o+l` $\rightarrow$ `ol`]

后面继续重新统计。常见合并序列:

[`ol`, `old`, `ug`, `olde`, `hug`]

新词:

- `hold` $\rightarrow$ [`h`, `old`]
- `oldest` $\rightarrow$ [`olde`, `s`, `t`]
- `mug` $\rightarrow$ [`[unk]`, `ug`]

中文总结:

BPE 题最容易错在“没有每轮重新统计 pair”。每次 merge 后，语料切分都变了，下一轮必须基于新的切分重新数。

Type B: Language Model and n-gram / 语言模型与 n-gram 题

Likely Question / 可能题目

What is a language model? Derive the chain rule factorization. Explain the n-gram approximation and its limitations. Given a sentence, list certain bigrams/trigrams.

中文题目:

什么是语言模型？写出联合概率的链式分解。解释 n-gram 近似及其局限。给定句子，列出指定 bigram/trigram。

How to Solve / 解题步骤

1. 先写定义: 语言模型给词序列分配概率，也可以预测下一个词。
2. 写链式法则:

[$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_1, \dots, w_{t-1})$]

1. 写 n-gram 近似:

$[P(w_t|w_1, \dots, w_{t-1}) \approx P(w_t|w_{t-n+1}, \dots, w_{t-1})]$

1. 如果要列 bigram/trigram , 先 tokenization , 然后找目标词位置。
2. 解释局限: sparsity、zero probability、Markov assumption、 $(O(V^n))$ 。

Example / 例题

Sentence:

I am taking CS6493 this semester and studying NLP is really fascinating

List all trigrams where the third word is CS6493 or NLP .

Answer / 参考答案

先分词:

$[[I, am, taking, CS6493, this, semester, and, studying, NLP, is, really, fascinating]]$ CS6493 前两个词是 am ,
taking : $[(am, taking, CS6493)]$ NLP 前两个词是 and , studying : $[(and, studying, NLP)]$

中文解释:

trigram 是连续 3 个词。如果题目说“第三个词是 X”, 就找 X 前面两个词, 加上 X 本身。

Type C: Cross-Entropy and Perplexity / 交叉熵与困惑度题

Likely Question / 可能题目

Given predicted probabilities and the true label, compute cross-entropy. Given average cross-entropy, compute perplexity. Explain why lower perplexity is better.

中文题目:

给定预测概率和真实类别, 计算交叉熵。给定平均交叉熵, 计算困惑度。解释为什么困惑度越低越好。

How to Solve / 解题步骤

1. 找真实类别对应的预测概率 (p_k)。
2. 交叉熵:

$[L = -\log p_k]$

1. 如果有多个 token , 先算每个 token 的 $(-\log p)$, 再取平均。
2. 困惑度:

$[PP = e^L]$

1. 解释: loss 越小, 模型给正确答案概率越高; perplexity 越小, 模型越不困惑。

Example / 例题

The correct next-token probabilities are:

[0.5, \quad 0.25, \quad 0.1]

Compute average cross-entropy and perplexity.

Answer / 参考答案

[$L = -\frac{1}{3}(\log 0.5 + \log 0.25 + \log 0.1)$] [$= -\frac{1}{3}(-0.693 - 1.386 - 2.303) = 1.461$]

PP = $e^{1.461} \approx 4.31$]

中文解释:

这个模型平均每一步的困惑度约等于在 4.31 个候选词中选择。困惑度越低，说明模型越能预测真实下一个词。

Type D: One-Hot and Word Embedding / one-hot 与词向量题

Likely Question / 可能题目

Construct one-hot vectors, compute dot products, explain why one-hot cannot represent semantic similarity, and compare one-hot with word embeddings.

中文题目:

构造 one-hot 向量，计算内积，解释为什么 one-hot 不能表示语义相似度，并比较 one-hot 和词向量。

How to Solve / 解题步骤

1. 按题目给定顺序或字典序排列词表。
2. 每个词写成长度为 $(|V|)$ 的向量。
3. 不同词 one-hot 内积为 0。
4. 解释问题: 所有不同词都同样不相似，不能表示 `cat` 和 `dog` 更接近。
5. 说明 embedding 是低维稠密向量，可学习语义相似度。

Example / 例题

Vocabulary:

[$V = \{\text{cat}, \text{dog}, \text{run}, \text{runs}\}$]

Lexicographic order. Write one-hot vectors and compute $(\text{cat}^T \text{dog})$.

Answer / 参考答案

[$\text{cat} = [1, 0, 0, 0]^T$] [$\text{dog} = [0, 1, 0, 0]^T$] [$\text{run} = [0, 0, 1, 0]^T$] [$\text{runs} = [0, 0, 0, 1]^T$] [$\text{cat}^T \text{dog} = [1, 0, 0, 0] \cdot [0, 1, 0, 0] = 0$]

中文解释:

one-hot 向量之间只要不是同一个词，内积就是 0。因此它无法表达词之间的语义相似性，也无法自动知道 `run` 和 `runs` 有词形关系。

Type E: Word2Vec, Negative Sampling, GloVe / 词向量训练题

Likely Question / 可能题目

Explain CBOW and Skip-gram. Why is full softmax slow? How does negative sampling improve efficiency? Compare Word2Vec and GloVe.

中文题目:

解释 CBOW 和 Skip-gram。为什么 full softmax 很慢？负采样如何提高效率？比较 Word2Vec 和 GloVe。

How to Solve / 解题步骤

1. CBOW: 上下文预测中心词。
2. Skip-gram: 中心词预测上下文。
3. full softmax 要对整个词表归一化，复杂度 $O(|V|)$ 。
4. negative sampling 只更新正样本和 (K) 个负样本，复杂度 $O(K)$ 。
5. GloVe 使用全局共现矩阵，目标是让词向量内积接近 $(\log X_{ij})$ 。

Example / 例题

Why does skip-gram train slowly on a large corpus, and how can negative sampling help?

Answer / 参考答案

Skip-gram 用中心词预测上下文词。如果使用完整 softmax:

$$[P(o|c) = \frac{\exp(uo^T v_c)}{\sum_{w \in V} \exp(uw^T v_c)}]$$

分母需要遍历整个词表，所以每次更新复杂度是 $O(|V|)$ 。词表很大时训练非常慢。

负采样只更新一个正样本和 (K) 个负样本:

$$[-\log \sigma(uo^T v_c) + \sum_{k=1}^K \log \sigma(-u_k^T v_c)]$$

复杂度变成 $O(K)$ ，而 $(K \ll |V|)$ ，所以训练更快。

Type F: Attention and Transformer / Attention 与 Transformer 题

Likely Question / 可能题目

Write the scaled dot-product attention formula. Explain Q/K/V. Explain why divide by $(\sqrt{d_k})$.

Write a causal mask. Analyze what happens if $(W^Q=W^K)$.

中文题目:

写出 scaled dot-product attention 公式。解释 Q/K/V。解释为什么除以 $(\sqrt{d_k})$ 。写 causal mask。分析如果 $(W^Q=W^K)$ 会怎样。

How to Solve / 解题步骤

1. 写公式:

$$[\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V]$$

1. 解释:

- Q: query, 要找什么
 - K: key, 被匹配的索引特征
 - V: value, 真正返回的内容
2. 除以 ($\sqrt{d_k}$): 防止 dot product 太大导致 softmax 过尖。
3. mask: 未来位置写 ($-\infty$), 当前和过去位置写 0。
4. ($W^Q=W^K$): ($S=QK^T=ZZ^T$), score matrix 对称, 限制方向性关系。

Example / 例题

Write the (4×4) causal mask for decoder self-attention.

Answer / 参考答案

$$[M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}]$$

中文解释:

第一行表示第 1 个 token 只能看自己; 第二行可以看第 1、2 个; 第三行可以看第 1、2、3 个; 第四行可以看所有过去和当前 token。mask 必须在 softmax 前加, 因为未来位置要变成概率 0, 并且不能影响分母。

Type G: BERT vs GPT / 预训练模型比较题

Likely Question / 可能题目

Compare BERT and GPT. Which is more suitable for NLU? Which is more suitable for NLG? Why?

中文题目:

比较 BERT 和 GPT。哪个更适理解任务? 哪个更适合生成任务? 为什么?

How to Solve / 解题步骤

1. 从结构比较: BERT encoder-only, GPT decoder-only。
2. 从 attention 比较: BERT bidirectional, GPT causal。
3. 从预训练目标比较: BERT masked LM, GPT next-token prediction。
4. 从任务比较: BERT 适合 NLU, GPT 适合 NLG。

Answer / 参考答案

BERT is encoder-only and uses bidirectional self-attention. It can see both left and right context, so it is suitable for understanding tasks such as classification, NER, and extractive QA.

中文: BERT 是 encoder-only, 能看双向上下文, 因此适理解任务, 例如文本分类、命名实体识别和抽取式问答。

GPT is decoder-only and uses causal masked attention. It predicts tokens from left to right, so it is suitable for generation tasks such as text completion, dialogue, and summarization.

中文: GPT 是 decoder-only, 只能从左到右看历史 token, 因此适合生成任务, 例如续写、对话和摘要生成。

Type H: NLU/NLG and Classification / 理解生成与分类题

Likely Question / 可能题目

Explain NLU vs NLG and classify tasks as binary, single-label multi-class, multi-label, or ordinal.

中文题目:

解释 NLU 和 NLG, 并判断任务属于二分类、单标签多分类、多标签分类还是有序分类。

How to Solve / 解题步骤

1. NLU: 理解或抽取意义。
2. NLG: 生成自然语言。
3. 分类判断:
 - 两类 -> binary
 - 多类但只选一个 -> single-label multi-class
 - 可同时多个标签 -> multi-label
 - 标签有顺序 -> ordinal

Example / 例题

Classify these:

1. spam vs legitimate email
2. news article assigned to exactly one section
3. paper tagged with all applicable categories
4. review rating from poor to excellent

Answer / 参考答案

1. spam vs legitimate: binary classification
2. exactly one news section: single-label multi-class classification
3. all applicable paper tags: multi-label classification
4. ordered review rating: ordinal classification

中文:

判断分类题时，不要只看类别数量，还要看“是否只能选一个”和“类别是否有顺序”。

Type I: QA and SQuAD / 问答与 SQuAD 题

Likely Question / 可能题目

What is extractive QA? Why is SQuAD important? What additional capability does SQuAD 2.0 require?

中文题目：

什么是抽取式问答？为什么 SQuAD 是重要 benchmark？SQuAD 2.0 需要模型具备什么额外能力？

How to Solve / 解题步骤

1. Extractive QA: 答案是 passage 中的 span。
2. SQuAD 优点: 大规模、人工问题、Wikipedia passage、标准 EM/F1 评估。
3. SQuAD 2.0: 加入 unanswerable questions，模型必须会拒答。

Answer / 参考答案

Extractive QA means the answer is copied directly as a span from the given passage. In SQuAD-style QA, the model often predicts start and end positions.

中文: 抽取式问答就是答案直接来自文章中的一段连续文本，模型通常预测答案的起始和结束位置。

SQuAD is important because it contains large-scale human-written questions on real Wikipedia passages and provides standard evaluation.

SQuAD 2.0 is harder because it includes unanswerable questions. The model must know when no answer is supported by the passage.

Type J: Decoding and BLEU / 解码与 BLEU 题

Likely Question / 可能题目

Use greedy search or beam search to decode a sequence. Compute BLEU from n-gram precisions and brevity penalty.

中文题目：

使用 greedy search 或 beam search 解码序列。根据 n-gram precision 和 brevity penalty 计算 BLEU。

How to Solve / 解题步骤

Greedy:

1. 每一步选概率最大 token。
2. 路径概率相乘。

Beam:

1. 保留 top (k) 条部分序列。
2. 展开所有候选。
3. 重新排序并保留 top (k)。

BLEU:

1. 写 (P_1, P_2, P_3, P_4) 。
2. 算 (BP)。
3. 代入:

$$[\text{BLEU} = \text{BP} \cdot \exp \left(\sum_n w_n \log P_n \right)]$$

Example / 例题

Given:

$$[P_1=8/9, \ P_2=6/8, \ P_3=5/7, \ P_4=4/6, \ c=9, \ r=9]$$

Compute BLEU.

Answer / 参考答案

$$[\text{BP}=1] [\text{BLEU} = \exp \left(\frac{\ln(8/9) + \ln(6/8) + \ln(5/7) + \ln(4/6)}{4} \right) \approx 0.7506]$$

中文:

BLEU 中 n-gram precision 防止生成无关内容，brevity penalty 防止输出过短。

Type K: Dialogue System and RAG / 对话系统与 RAG 设计题

Likely Question / 可能题目

Design a task-oriented dialogue system or a RAG-based QA assistant. Explain modules and evaluation metrics.

中文题目:

设计任务型对话系统或基于 RAG 的问答助手。解释模块和评价指标。

How to Solve / 解题步骤

Task-oriented dialogue:

$$[\text{NLU} \rightarrow \text{DST} \rightarrow \text{Policy} \rightarrow \text{Backend/API} \rightarrow \text{NLG}]$$

RAG:

$$[\text{Documents} \rightarrow \text{Chunking} \rightarrow \text{Embedding} \rightarrow \text{VectorDB} \rightarrow \text{Retrieval}]$$

\rightarrow Reranking \rightarrow LLM \rightarrow Citation]

Example / 例题

Design a campus assistant that answers student questions using official documents.

Answer / 参考答案

Use a RAG system:

1. Collect official PDFs, web pages, and policy documents.
2. Split documents into chunks.
3. Embed chunks and store them in a vector database with metadata.
4. Embed student query.
5. Retrieve relevant chunks.
6. Rerank/filter evidence.
7. LLM generates answer using retrieved evidence only.
8. Provide citations.
9. If evidence is weak, abstain or ask clarification.

中文:

RAG 适合校园政策问答，因为政策经常更新，必须基于官方文档回答。检索可以提供最新证据，生成模型负责组织语言，引用和拒答机制可以降低幻觉和过时信息风险。

Type L: LoRA, ZeRO, Efficient Training / 高效训练题

Likely Question / 可能题目

Why does LLM training require so much memory? What is ZeRO? Why is LoRA parameter-efficient?

中文题目:

为什么训练大模型很占显存？ZeRO 是什么？为什么 LoRA 参数高效？

How to Solve / 解题步骤

1. 写 Adam fp16 显存组成:

$[2 \times \Psi \times \text{weights} + 2 \times \Psi \times \text{gradients} + 12 \times \Psi \times \text{optimizer} \times \text{states} = 16 \times \Psi]$

1. 说明 activation memory 还会额外占显存。
2. ZeRO: 分片 optimizer states、gradients、parameters。
3. LoRA: 冻结原始权重，只训练低秩更新。

Answer / 参考答案

For a model with (Ψ) parameters, mixed-precision Adam training stores fp16 weights, gradients, fp32 weight copy, momentum, and variance. Total memory for weights/gradients/optimizer states is about:

[16\Psi]

ZeRO reduces memory by partitioning optimizer states, gradients, and parameters across GPUs.

LoRA freezes the pretrained weight (W) and learns:

[$\Delta W = BA$]

where rank (r) is small. Therefore, it trains far fewer parameters than full fine-tuning.

中文:

LoRA 的核心是“原模型不动，只学一个低秩变化量”。所以它省显存、省训练成本，而且训练后可以把 LoRA 权重合并回原模型。

B. Question Type Recognition / 看到题目怎么判断题型

Type 1: Definition + Example / 定义加例子题

Typical wording:

- "Explain what is..."
- "What is the difference between..."
- "Give examples of..."

Answer structure:

1. One-sentence definition in English or Chinese.
2. Explain in course terminology.
3. Give one concrete example.
4. If it is a comparison question, list at least two differences.

Example:

Question: What is the distributional hypothesis?

Answer skeleton:

English: The distributional hypothesis states that words appearing in similar contexts tend to have similar meanings. It motivates word embedding methods because a word can be represented by the contexts in which it appears. For example, "hotel" and "motel" may appear near words like "room", "stay", and "booking", so their embeddings should be close.

中文: 分布式假设指“出现在相似上下文中的词通常具有相似含义”。它是词向量方法的基础，因为我们可以用一个词周围的上下文来表示这个词。例如 hotel 和 motel 都可能出现在 room、stay、booking 附近，所以它们的词向量应该接近。

Type 2: Calculation / 计算题

Typical wording:

- "Compute..."
- "Show your calculation..."
- "Write down the matrix..."
- "List all bigrams/trigrams..."
- "Using greedy/beam search..."
- "Compute BLEU..."

Answer rule:

Never only write final answer. You need:

1. Write the formula.
2. Write the intermediate values.
3. Substitute numbers.
4. Give final result.
5. Add one sentence interpretation if marks allow.

Most likely calculation topics:

- One-hot vector and dot product.
 - Embedding parameter count.
 - n-gram extraction.
 - BPE merge rules and tokenization.
 - Greedy and beam search path probability.
 - BLEU.
 - Attention mask matrix.
 - Precision/recall/F1.
 - LLM memory ($16 \times \Psi$).
-

Type 3: Model/System Design / 系统设计题

Typical wording:

- "Design a..."
- "What modules are needed..."
- "How does the system work..."
- "How to evaluate..."

Answer structure:

1. State the task goal.
2. Draw/list modules in order.

3. Explain each module using the scenario.
4. Add evaluation metrics.
5. Mention failure handling or limitations.

For task-oriented dialogue: [NLU \rightarrow DST \rightarrow Policy \rightarrow Backend/API \rightarrow NLG]

For RAG:

[Documents \rightarrow Chunking \rightarrow Embedding \rightarrow Vector \ DB \rightarrow Retrieval \rightarrow Reranking \rightarrow LLM \rightarrow Citation/Verification]

C. Must-Know Problem Solving Templates / 必背题型模板

C1. BPE Calculation Template / BPE 计算题模板

BPE is very likely because HW1 explicitly tested it.

What BPE does / BPE 做什么:

English: Byte Pair Encoding is a subword tokenization algorithm. It starts from characters and repeatedly merges the most frequent adjacent symbol pair. It reduces OOV problems because unseen words can be decomposed into known subwords.

中文: BPE 是一种子词分词方法。它从字符开始，每次合并语料中最频繁的相邻符号对。它能缓解 OOV，因为没见过的词也可以拆成已知子词。

Training steps / 训练步骤:

1. Pre-tokenize corpus into words and word frequencies.
2. Split each word into characters.
3. Count all adjacent symbol pairs, weighted by word frequency.
4. Select the most frequent pair.
5. Merge that pair everywhere in the corpus.
6. Add the merged symbol to vocabulary.
7. Repeat until reaching the target vocabulary size or target number of merges.

Important: Count adjacent pairs after each merge. Do not count arbitrary substrings.

中文重点: 每一轮只能统计“当前切分结果里的相邻 pair”，不能随便统计所有子串。每合并一次，语料的切分会变化，下一轮要重新数。

HW1 example:

Words:

[(old,10),(older,5),(oldest,8),(hug,8),(pug,4),(hugs,5)]

Base vocabulary:

[o,l,d,e,r,s,t,h,u,g,p]

Base vocab size is 11. If target vocab size is 16, number of merges:

[$16-11=5$]

Initial symbolized corpus:

[(o,l,d;10)] [(o,l,d,e,r;5)] [(o,l,d,e,s,t;8)] [(h,u,g;8)] [(p,u,g;4)] [(h,u,g,s;5)]

Round 1 pair counts:

- ((o,l)): $(10+5+8=23)$
- ((l,d)): $(10+5+8=23)$
- ((d,e)): $(5+8=13)$
- ((u,g)): $(8+4+5=17)$
- ((h,u)): $(8+5=13)$
- other pairs lower

The question statement says first merge is $(o+l=ol)$. If there is a tie, follow the question's given rule or the first pair in scan order.

After merge:

[(ol,d;10), (ol,d,e,r;5), (ol,d,e,s,t;8), (h,u,g;8), (p,u,g;4), (h,u,g,s;5)]

A consistent 5-merge sequence is:

1. $(o+l \rightarrow ol)$
2. $(ol+d \rightarrow old)$
3. $(u+g \rightarrow ug)$
4. $(old+e \rightarrow olde)$
5. $(h+ug \rightarrow hug)$

Depending on tie-breaking, the 4th/5th merge may swap, but you must show your counts and be consistent.

Final vocab includes:

[o,l,d,e,r,s,t,h,u,g,p,ol,old,ug,olde,hug]

Tokenizing new words / 用训练好的 BPE 分新词:

Use longest-match from left to right with learned vocabulary. If a character is not in base vocabulary, output [unk].

Examples:

- hold -> h old
- oldest -> olde s t
- older -> olde r
- pug -> p ug
- mug -> [unk] ug because m is not in base vocabulary
- huggingface -> hug g [unk] [unk] [unk] g [unk] [unk] [unk] e

Why `huggingface` has many `[unk]`: base vocabulary does not contain `i`, `n`, `f`, `a`, `c`.

C2. n-gram Extraction Template / n-gram 抽取题模板

If asked for bigrams where the second word is X:

1. Tokenize sentence.
2. Find X.
3. Take previous word + X.

If asked for trigrams where the third word is X:

1. Tokenize sentence.
2. Find X.
3. Take previous two words + X.

HW1 sentence: `I am taking CS6493 this semester and studying NLP is really fascinating`

Bigrams with second word `CS6493` or `NLP`:

[(taking,CS6493),(studying,NLP)]

Trigrams with third word `CS6493` or `NLP`:

[(am,taking,CS6493),(and,studying,NLP)]

Explanation question:

- n-gram cannot generalize to unseen combinations because it only counts observed sequences.
 - Markov assumption limits context to (n-1) words.
 - Possible n-grams grow as $O(V^n)$, causing sparsity.
-

C3. Greedy / Beam Search Template / 解码题模板

Greedy:

1. Start from `<s>`.
2. At each step, choose the token with maximum conditional probability.
3. Multiply selected probabilities.
4. Stop at `</s>`.

Beam search:

1. Keep top (k) partial sequences.
2. Expand every kept sequence by possible next tokens.
3. Multiply path probabilities or add log probabilities.
4. Keep top (k) again.
5. Continue until ending condition.

Key sentence:

English: Beam search can outperform greedy search because it keeps several promising partial sequences instead of committing to a locally best token.

中文: beam search 比 greedy 更可能找到全局更优序列, 因为它不会每一步只押注一个局部最优选择。

C4. BLEU Calculation Template / BLEU 计算题模板

Formula:

$$[\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log P_n\right)]$$

Uniform weights:

$$[w_n = \frac{1}{N}]$$

Brevity penalty:

$$[\text{BP} = \begin{cases} 1, & c > r \\ \exp(1-r/c), & c \leq r \end{cases}]$$

Steps:

1. Compute (P_1, P_2, P_3, P_4) .
2. Compute (c) and (r) .
3. Compute (BP) .
4. Plug into formula.

$$\text{HW2 values: } [P_1=8/9, P_2=6/8, P_3=5/7, P_4=4/6, c=9, r=9] [\text{BP}=1] [$$

$$\text{BLEU} = \exp\left(\frac{\ln(8/9) + \ln(6/8) + \ln(5/7) + \ln(4/6)}{4}\right) \approx 0.7506]$$

中文解释:

- n-gram precision 惩罚乱生成或无关内容。
- brevity penalty 惩罚过短输出。

C4.5 Cross-Entropy Calculation Template / 交叉熵计算题模板

Use this template whenever the question gives a true class and predicted probabilities.

Classification cross-entropy:

$$[L = -\sum_{i=1}^C y_i \log p_i]$$

If the gold label is class (k) :

$$[L = -\log p_k]$$

Steps:

1. Identify the correct class (k) .
2. Find the model probability assigned to that class, (p_k) .
3. Compute $(-\log p_k)$.
4. If there are multiple samples/tokens, average or sum depending on the question.

Example:

Gold label is class 3. Prediction is:

[[0.2,0.1,0.6,0.1]]

Correct class probability:

[p₃=0.6]

Cross-entropy:

[$L = -\log(0.6) \approx 0.511$]

Language modeling:

[$L = -\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{-})$ Perplexity:

[$PP = e^L$]

中文速记:

交叉熵就是“正确答案概率取 log 再加负号”。正确答案概率越高，loss 越低；正确答案概率越低，loss 越高。语言模型的每个 token 预测都是一次词表分类，所以也用交叉熵。

C5. Attention and Mask Template / Attention 推导题模板

Standard formula:

[$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$] [$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$]

If ($W^Q = W^K$):

[$Q = K = XW = Z$] [$S = QK^T = ZZ^T$]

Therefore:

- (S) is symmetric.
- This limits asymmetric directional attention.

If ($V=K$):

- key and value roles are forced to be the same.
- model cannot separately learn "where to attend" and "what information to return".

Causal mask for (n=4): [$\begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$]

Why before softmax:

English: Future positions must receive probability zero and must not enter the softmax denominator.

中文: 未来位置必须概率为 0，并且不能参与 softmax 分母归一化，所以 mask 要在 softmax 前加。

C6. Task-Oriented Dialogue Template / 任务型对话设计模板

For CRB or any task chatbot:

1. **Domain classification:** decide broad service domain.
2. **Intent detection:** decide user's specific goal.
3. **Slot filling:** extract required fields.
4. **Dialogue state tracking:** remember filled and missing slots.
5. **Dialogue policy:** decide next action.
6. **Backend/API:** validate and submit.
7. **NLG:** generate user-facing response.
8. **Evaluation:** intent accuracy, slot F1, task success rate, average turns, user satisfaction.

CRB slots:

- patient name
 - ID/HKID
 - test result
 - test date
 - phone number
 - address
 - test type
 - symptom onset date
-

C7. RAG Design Template / RAG 设计题模板

Use this for campus assistant or policy QA:

1. Collect official documents.
2. Split into chunks.
3. Embed chunks.
4. Store in vector DB with metadata.
5. Convert query to embedding.
6. Retrieve top-k chunks.
7. Rerank/filter.
8. Generate answer grounded in evidence.
9. Add citations.
10. If evidence is weak, abstain or ask clarification.

Why RAG helps:

- updates knowledge without retraining
- reduces hallucination
- provides citations

- handles domain-specific knowledge
-

0. Exam Map / 考试知识地图

High-frequency themes:

- **Basic NLP pipeline / 基础流程**: tokenization, stop-word removal, stemming, lemmatization, vectorization.
- **Language models / 语言模型**: n-gram, Markov assumption, neural LM, RNN/LSTM, perplexity.
- **Word representation / 词表示**: one-hot, distributional hypothesis, word2vec, GloVe, ELMo.
- **Transformer / 预训练模型**: attention, self-attention, multi-head attention, positional encoding, BERT vs GPT.
- **NLU tasks / 理解任务**: text classification, QA, SQuAD, BiDAF, metrics.
- **NLG tasks / 生成任务**: machine translation, seq2seq, decoding, beam search, BLEU, dialogue systems.
- **LLM topics / 大模型专题**: scaling law, emergent abilities, prompting, CoT, alignment, RLHF/DPO, agents, PEFT, RAG.

闭卷答题建议:

- 先写 **definition** / 定义, 再写 **why** / 为什么, 最后写 **example** / 例子。
 - 计算题必须写公式、代入、最后结果。
 - 比较题用表格或分点: advantage, limitation, use case.
-

Lecture 1 - Introduction to NLP / NLP 导论

What Is NLP?

English: Natural Language Processing is a branch of AI that enables computers to learn, process, understand, manipulate, and generate human language.

中文: 自然语言处理是人工智能的一个分支, 目标是让计算机能够学习、处理、理解、操作和生成自然语言, 从而与人类进行语言交互。

Main NLP Pipeline

English: A typical NLP pipeline converts raw text into machine-processable representations and then applies models for understanding or generation.

中文: 典型 NLP 流程是把原始文本转换为机器可处理的表示, 再使用模型完成理解或生成任务。

Key preprocessing steps:

- **Tokenization / 分词**: split text into words, characters, or subwords.
- **Stop-word removal / 停用词去除**: remove frequent low-information words such as "the", "is".
- **Stemming / 词干提取**: reduce words to rough stems, e.g. `playing -> play`.
- **Lemmatization / 词形还原**: convert words to dictionary forms using linguistic knowledge, e.g. `went -> go`.
- **Vectorization / 向量化**: convert text to numbers, e.g. one-hot, bag-of-words, TF-IDF, embeddings.

NLP Challenges

English: Natural language is ambiguous, context-dependent, sparse, informal, multilingual, and constantly changing.

中文: 自然语言存在歧义、上下文依赖、数据稀疏、口语化、多语言差异和持续变化等问题。

Examples:

- **Lexical ambiguity / 词义歧义**: "bank" can mean river bank or financial institution.
- **Syntactic ambiguity / 句法歧义**: "I saw the man with a telescope."
- **Coreference / 指代**: deciding what "it", "he", "this" refer to.
- **Long-range dependency / 长距离依赖**: words far apart may determine meaning.

Historical Development

- Rule-based NLP -> statistical NLP -> neural NLP -> pretrained models -> large language models.
- 规则系统 -> 统计方法 -> 神经网络方法 -> 预训练模型 -> 大语言模型。

Important milestones:

- **1997 LSTM**: helps RNNs model longer dependencies.
- **2013 word2vec**: efficient dense word embeddings.
- **2015 attention**: lets models focus on relevant input parts.
- **2017 Transformer**: attention-based architecture.
- **BERT/GPT/LLMs**: large-scale pretraining and prompting.

Lecture 2 - Language Models / 语言模型

Definition

English: A language model is a probability distribution over word sequences. For a sequence $(x^{(1)}, \dots, x^{(T)})$, it assigns:

$$[P(x^{(1)}, x^{(2)}, \dots, x^{(T)})]$$

Using the chain rule:

$$[P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1})]$$

中文: 语言模型是对词序列的概率分布。它给一句话整体赋予概率，也可以理解为根据前文预测下一个词。

Applications

- Predictive typing / 输入法预测
- Speech recognition / 语音识别
- Spelling and grammar correction / 拼写和语法纠错
- Machine translation / 机器翻译
- Summarization / 摘要
- Dialogue / 对话
- Text generation / 文本生成

n-gram Language Model

English: n-gram models use the Markov assumption: the next word depends only on the previous (n-1) words.

$$[P(x_t | x_1, \dots, x_{t-1}) \approx P(x_t | x_{t-n+1}, \dots, x_{t-1})]$$

中文: n-gram 模型假设当前词只依赖前面 (n-1) 个词，而不是整个历史。这降低了建模难度，但会丢失长距离上下文。

Examples:

- **Unigram:** $(P(x_t))$
- **Bigram:** $(P(x_t | x_{t-1}))$
- **Trigram:** $(P(x_t | x_{t-2}, x_{t-1}))$

n-gram Limitations

- **Sparsity / 稀疏性:** many n-grams never appear in training data.
- **Zero probability / 零概率:** unseen sequences may get probability 0.
- **Short context / 上下文短:** cannot capture long-distance dependency.
- **Parameter explosion / 参数爆炸:** possible n-grams grow as $(O(V^n))$.

Neural Language Models

English: Neural LMs replace sparse count tables with learned dense representations and neural networks, improving generalization.

中文: 神经语言模型使用词向量和神经网络替代表格计数，能在相似上下文之间泛化。

Training objective:

[$\mathcal{L} = -\sum_{t=1}^T \log P(x_t | x_{<t})$] This is cross-entropy / negative log-likelihood.

RNN Language Model

English: RNN processes a sequence step by step and updates a hidden state:

[$h_t = f(W_{xx}x_t + W_{hh}h_{t-1} + b)$]

Then predicts:

[$P(x_{t+1} | x_{\leq t}) = \text{softmax}(W_o h_t)$]

中文: RNN 逐词读取序列，用隐藏状态 (h_t) 保存历史信息，再预测下一个词。

Advantages:

- Handles variable-length input.
- Reuses parameters across time.
- Model size does not grow with sequence length.

Limitations:

- Vanishing/exploding gradients.
- Hard to learn very long dependencies.
- Sequential computation is slow.

LSTM and GRU

English: LSTM uses gates and a memory cell to preserve information over long spans. GRU is a simpler gated recurrent unit.

中文: LSTM 通过输入门、遗忘门、输出门和记忆单元缓解梯度消失，更适合长期依赖。GRU 结构更简单，计算更轻。

Perplexity

English: Perplexity is the standard evaluation metric for language models. Lower is better.

For a sequence of length (T):

[$PP(W) = P(w_1, \dots, w_T)^{-1/T}$]

Equivalently, if average negative log-likelihood is (H):

[$PP = \exp(H)$]

中文: 困惑度衡量模型对文本的“不确定程度”。困惑度越低，模型越能预测真实文本。

Interpretation:

- ($PP=10$): model is roughly as uncertain as choosing among 10 equally likely words at each step.
 - Lower perplexity usually means better language modeling, but not always better downstream task performance.
-

Lecture 3 - Word Embedding / 词向量

One-Hot Representation

English: One-hot represents each word as a vocabulary-sized vector with one 1 and all other entries 0.

中文: one-hot 用一个长度等于词表大小的向量表示词，只有该词对应位置为 1，其余为 0。

Limitations:

- Sparse and high-dimensional.
- All different words are orthogonal: $(\mathbf{e}_i^T \mathbf{e}_j = 0)$.
- Cannot encode semantic similarity, e.g. `cat` and `dog` are not closer than `cat` and `run`.
- Cannot capture morphology, e.g. `run` and `runs` are unrelated vectors.

Distributional Hypothesis

English: Words that occur in similar contexts tend to have similar meanings.

中文: 分布式假设: 出现在相似上下文中的词往往具有相似含义。

This is the foundation of word embeddings.

Word Embedding

English: A word embedding is a dense, low-dimensional vector learned from data. Similar words are close in vector space.

中文: 词向量是稠密、低维、实数值的词表示。语义相近的词在向量空间中距离更近。

Advantages:

- Dense and efficient.
- Captures semantic/syntactic similarity.
- Supports vector arithmetic and nearest-neighbor search.
- Better generalization than one-hot.

word2vec: CBOW and Skip-Gram

CBOW / Continuous Bag of Words

English: Predict center word from surrounding context words.

中文: 用上下文预测中心词。

$[P(w_t | w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c})]$

Skip-gram

English: Predict surrounding context words from the center word.

中文: 用中心词预测上下文词。

$$[P(w_{t+j}|w_t), \quad -c \leq j \leq c, \quad j \neq 0]$$

Comparison:

- CBOW is faster and works well for frequent words.
- Skip-gram is better for rare words but slower.

Skip-Gram Training Problem

Full softmax over vocabulary is expensive:

$$[P(o|c) = \frac{\exp(uo^{\top}vc)}{\sum_{w \in V} \exp(uw^{\top}vc)}]$$

Cost is $(O(|V|))$ per update, too slow for large vocabularies.

Negative Sampling

English: Instead of updating all vocabulary words, update the positive context word and (K) sampled negative words.

中文: 负采样不再对整个词表做 softmax，而是只更新真实上下文词和 (K) 个随机负样本，因此训练更快。

Objective for a positive pair $((c,o))$:

$$[\log \sigma(uo^{\top}vc) + \sum_{k=1}^K \mathbb{E}_{wk \sim P_n(w)} \log \sigma(-u{wk}^{\top}vc)]$$

Noise distribution often uses:

$$[P_n(w) \propto \text{count}(w)^{3/4}]$$

Key point for exam:

- Positive pair should have high dot product.
- Negative pairs should have low dot product.
- Complexity becomes $(O(K))$, where $(K \ll |V|)$.

Hierarchical Softmax

English: Use a binary tree to reduce softmax cost from $(O(|V|))$ to $(O(\log |V|))$.

中文: 层次 softmax 使用二叉树表示词表，把预测词的概率分解为树路径上的二分类概率，计算复杂度从 $(O(|V|))$ 降到 $(O(\log |V|))$ 。

It often works well for infrequent words.

Subsampling Frequent Words

English: Very frequent words such as "the" and "of" dominate training but carry little semantic information. Subsampling drops them with higher probability.

中文: 高频词会占据大量训练样本但语义信息少，因此可以随机丢弃一部分高频词来加速并提升表示质量。

GloVe

English: GloVe combines global corpus co-occurrence statistics with local context-window learning.

中文: GloVe 结合全局共现统计和局部上下文窗口，是 global statistics + local context 的方法。

Objective:

$$[J = \sum_{i,j} f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2]$$

Where:

- (X_{ij}) : co-occurrence count.
- $(f(X_{ij}))$: weighting function.
- $(w_i, \tilde{w}_j)$: word and context embeddings.

Contextualized Embeddings: ELMo

English: ELMo gives different embeddings for the same word in different contexts.

中文: ELMo 解决一词多义问题。同一个词在不同句子中会得到不同的上下文文化表示。

Example:

- "stick" in "wooden stick" vs "stick to the plan" should have different representations.

ELMo uses bidirectional LSTM language models and combines internal layers for downstream tasks.

Lecture 4 - Transformer and Pretraining/Fine-Tuning / Transformer 与预训练微调

Why Attention?

English: Attention lets a model dynamically focus on relevant parts of the input rather than compressing everything into a single fixed vector.

中文: Attention 让模型在生成或理解时动态关注输入中最相关的部分，而不是把整句压缩成一个固定向量。

Benefits:

- Handles long-distance dependency better than RNNs.
- Connects all positions directly.
- More parallelizable.
- Attention weights can be partially interpretable.

General Attention

Given query (q), keys (k_i), values (v_i):

$$[s_i = \text{similarity}(q, k_i)] \quad [a_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}] \quad [z = \sum_i a_i v_i]$$

中文解释: query 表示“我要找什么”，key 表示“每个位置有什么索引特征”，value 表示“真正要取出的内容”。先计算 query 和 key 的相似度，再 softmax 成权重，最后对 value 加权求和。

Scaled Dot-Product Attention

$$[\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V]$$

Why divide by $(\sqrt{d_k})$?

English: To prevent dot products from becoming too large when dimension is high, which would make softmax too sharp and gradients small.

中文: 除以 $(\sqrt{d_k})$ 是为了避免维度高时点积过大，导致 softmax 过于尖锐、梯度变小。

Self-Attention

English: In self-attention, Q, K, and V all come from the same input sequence.

$$[Q = XW^Q, \quad K = XW^K, \quad V = XW^V]$$

中文: self-attention 中，每个词都可以关注同一句子里的其他词，因此能直接建模词与词之间的依赖关系。

Multi-Head Attention

English: Multi-head attention runs several attention heads in parallel, allowing the model to capture different relation types.

中文: 多头注意力用多个 attention head 并行学习不同关系，例如语法关系、指代关系、局部搭配等。

$$[\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)] \quad [\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O]$$

Positional Encoding

English: Transformer has no recurrence or convolution, so positional information must be added.

中文: Transformer 本身没有顺序结构，因此必须加入位置编码，让模型知道词序。

Sinusoidal positional encoding:

$$[PE(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d})] \quad [PE(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{2i/d})]$$

Transformer Encoder vs Decoder

Encoder:

- Self-attention over full input.
- Feed-forward network.

- Used by BERT-like models.

Decoder:

- Masked self-attention to prevent seeing future tokens.
- Cross-attention to encoder output in seq2seq models.
- Used by GPT-like autoregressive generation.

Masked Self-Attention

English: Decoder uses a causal mask so token (t) can only attend to positions ($\leq t$).

中文: 解码器使用 causal mask，保证当前位置不能看未来词，否则训练和生成不一致。

Mask is added before softmax:

$$[A = \text{softmax}(\frac{QK^T + M}{\sqrt{d_k}})]$$

Forbidden positions receive ($-\infty$), so their probability becomes 0.

BERT vs GPT

Aspect	BERT	GPT
Architecture	Encoder-only	Decoder-only
Attention	Bidirectional self-attention	Causal masked self-attention
Pretraining	Masked LM, NSP in original BERT	Autoregressive next-token prediction
Strength	Understanding tasks	Generation tasks
Example tasks	Classification, NER, QA	Text generation, dialogue, completion

中文总结:

- BERT 看双向上下文，适合理解。
- GPT 从左到右预测下一个 token，适合生成。

Pretraining and Fine-Tuning

English: Pretraining learns general language knowledge from large unlabeled corpora. Fine-tuning adapts the model to a downstream supervised task.

中文: 预训练在大规模无标注文本上学习通用语言知识；微调用少量标注数据把模型适配到具体任务。

Lecture 5 - NLU Tasks / 自然语言理解任务

NLU vs NLG

NLU / Natural Language Understanding

English: Map text input to meaning, labels, spans, structures, or decisions.

中文: NLU 关注理解文本，把语言输入映射为含义、标签、实体、答案或结构化表示。

Examples:

- Text classification
- Sentiment analysis
- Intent detection
- Named entity recognition
- Extractive QA

NLG / Natural Language Generation

English: Produce fluent natural language from data, meaning, or context.

中文: NLG 关注生成文本，例如翻译、摘要、对话生成。

Text Classification

Definition:

$[h: \mathcal{D} \rightarrow C]$

where ($\mathcal{D}$) is a set of text items and (C) is a finite set of classes.

中文: 文本分类是给句子、段落、文档等文本单元分配类别标签。

Types:

- **Binary classification / 二分类:** spam vs legitimate.
- **Single-label multi-class / 单标签多分类:** one news category among many.
- **Multi-label classification / 多标签分类:** a paper can have multiple topics.
- **Ordinal classification / 有序分类:** ratings from poor to excellent.

Hard vs Soft Classification

- **Hard classification:** output final class label.
- **Soft classification:** output probability or score for each class.

中文:

- hard 输出类别。
- soft 输出每个类别的概率/置信度。

Common Text Classification Models

- Bag-of-words + logistic regression/SVM.
- CNN/RNN/TextRNN.
- Pretrained models such as BERT/GPT.
- Fine-tuning PLMs for domain-specific tasks.

Classification Metrics

For binary classification:

$$\left[\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \right] \left[\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \right] \left[\text{F1} = \frac{2\text{PR}}{\text{P} + \text{R}} \right] \left[\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \right]$$

中文解释:

- **Precision:** 预测为正的样本中有多少是真的正。
- **Recall:** 所有真实正样本中找回了多少。
- **F1:** precision 和 recall 的调和平均。
- **Accuracy:** 总体预测正确比例。

Question Answering

Types:

- **Factoid QA / 事实型问答:** asks for facts.
- **IR-based QA / 信息检索问答:** retrieve relevant documents/passages.
- **Knowledge-based QA / 知识库问答:** query structured knowledge base.
- **Reading comprehension / 阅读理解:** answer based on a given passage.

SQuAD

SQuAD 1.1

English: A reading comprehension benchmark where answers are text spans in Wikipedia passages.

中文: SQuAD 1.1 是抽取式阅读理解数据集，答案是文章中的连续文本片段。

SQuAD 2.0

Adds unanswerable questions. Models must decide whether the passage contains enough evidence.

中文: SQuAD 2.0 加入无答案问题，模型不仅要回答，还要判断是否应该拒答。

Extractive QA Model Output

For passage tokens $(x_1, \dots, x_n)$, model predicts:

$$[P\{start\}(i), \quad P\{end\}(j)]$$

Answer span is $((i, j))$.

QA Metrics

- **Exact Match (EM):** predicted answer exactly matches gold answer.
- **F1:** token-level overlap between prediction and reference.

中文:

- EM 要求完全一致。
- F1 把答案当作 token bag , 计算 precision/recall 的调和平均。

BiDAF

English: BiDAF uses bidirectional attention flow: context-to-query and query-to-context attention.

中文: BiDAF 使用双向注意力, 包括上下文关注问题、问题关注上下文, 增强阅读理解中的对齐能力。

Lecture 6 - NLG Tasks / 自然语言生成任务

NLG Tasks

- Machine translation / 机器翻译
- Summarization / 摘要
- Paraphrasing / 改写
- Dialogue generation / 对话生成
- Report generation / 报告生成

Machine Translation

English: Translate a source sentence (x) into a target sentence (y).

$[\hat{y} = \arg \max_y P(y|x)]$

中文: 机器翻译把源语言句子 (x) 转换为目标语言句子 (y), 核心是最大化条件概率 ($P(y|x)$)。

SMT vs NMT

Statistical Machine Translation / 统计机器翻译

- Uses translation model and language model.
- Requires alignment as latent variable.
- Often phrase-based and feature-based.

Neural Machine Translation / 神经机器翻译

- Uses encoder-decoder neural networks.
- Directly models ($P(y|x)$).
- Became dominant after seq2seq and attention.

Seq2Seq

English: Encoder reads source sentence and produces hidden states; decoder generates target sentence one token at a time.

中文: 编码器读取源句，解码器根据编码信息逐词生成目标句。

Seq2seq is a conditional language model:

$P(y|x) = \prod_{t=1}^T P(y_t | y_{<T})$ {Decoding

Greedy Search

Pick the highest-probability token at each step.

中文: 每一步选当前概率最大的词，速度快但容易陷入局部最优。

Beam Search

Keep top (k) partial hypotheses at each step.

中文: beam search 保留 (k) 个候选序列，能比 greedy 更接近全局最优，但计算更贵。

Length Normalization

Raw probability favors short sequences because multiplying probabilities makes longer sequences smaller. Length normalization reduces this bias.

中文: 原始概率偏向短句，因为概率连乘会越来越小；长度归一化用于减轻短输出偏置。

BLEU

English: BLEU evaluates generated text, especially translation, using modified n-gram precision and brevity penalty.

中文: BLEU 用修正 n-gram precision 和 brevity penalty 评估生成文本，尤其是机器翻译。

$$[\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log P_n\right)]$$

Brevity penalty:

$$[\text{BP} = \begin{cases} 1, & c > r \\ \exp(1 - r/c), & c \leq r \end{cases}]$$

where:

- (c): candidate length.
- (r): reference length.
- (P_n): modified n-gram precision.

Key interpretation:

- n-gram precision discourages irrelevant/hallucinated text.
- BP discourages overly short outputs.

Dialogue Systems

Types:

- **Task-oriented dialogue / 任务型对话:** complete a task, e.g. booking, reporting, customer service.
- **Chitchat dialogue / 闲聊:** open-domain social conversation.

Task-oriented pipeline:

1. NLU: domain classification, intent detection, slot filling.
2. Dialogue state tracking: maintain current user goal and filled slots.
3. Dialogue policy: decide next system action.
4. NLG: generate natural language response.

中文:

任务型对话系统先理解用户输入，再维护对话状态，决定下一步动作，最后生成回复。

Lecture 7 - Large Language Models / 大语言模型

What Is an LLM?

English: A large language model is a neural language model trained on massive text corpora with large parameter count and compute, usually based on Transformer architecture.

中文: 大语言模型是在海量文本上训练、参数规模和计算量都很大的神经语言模型，通常基于Transformer。

Scaling Law

English: Scaling laws describe predictable improvements in loss/perplexity as model size, data size, and compute increase.

中文: scaling law 指模型规模、数据规模和计算量增加时，loss/perplexity 往往会按规律下降。

Resources:

- Model size / 参数量
- Dataset size / 训练 token 数
- Compute / 计算量

Chinchilla Insight

English: For a fixed compute budget, it may be better to train a smaller model on more tokens than a much larger model on too few tokens.

中文: 在固定计算预算下，模型参数和训练数据要平衡；不是参数越大越好，如果数据不足会欠训练。

Emergent Abilities

English: Some abilities appear only when models reach sufficient scale, such as in-context learning and chain-of-thought reasoning.

中文: 涌现能力是模型达到一定规模后突然表现出来的能力，例如上下文学习、复杂推理、CoT。

Paradigm Shift

Pretraining + fine-tuning:

- Update model parameters for each downstream task.

Prompting:

- Keep model frozen and solve tasks through natural-language instructions/examples.

中文:

过去是预训练后微调；LLM 时代常通过 prompt 直接调用模型能力，不一定更新参数。

Alignment

English: Alignment means making model behavior better match human intent, preferences, and values.

中文: 对齐是让模型输出更符合人类意图、偏好和价值规范。

Why needed:

- LMs predict likely text, not necessarily helpful/truthful/safe text.
- Models can be fluent but wrong or harmful.

RLHF / InstructGPT

Typical RLHF pipeline:

1. Supervised fine-tuning on instruction-response data.
2. Train reward model from human preference comparisons.
3. Optimize policy using reinforcement learning, often PPO.

中文:

RLHF 先用人工示范做监督微调，再训练奖励模型，最后用强化学习让模型更偏向人类喜欢的回答。

Lecture 8 - Prompting and Alignment / Prompt 与对齐

Prompt

English: A prompt is text inserted into the model input to reformulate a task as language modeling.

中文: prompt 是加入模型输入的文字，用来把下游任务改写成语言模型可以直接处理的形式。

Example:

Sentiment classification:

No reason to watch. It was ____

Label words:

- positive -> `great`
- negative -> `terrible`

Prompt Components

- Instruction / 指令
- Context / 上下文
- Examples / 示例
- Question / 问题
- Output format / 输出格式

Prompt Shapes

- **Cloze prompt:** fill in a blank.
- **Prefix prompt:** continue a prefix.

中文:

- cloze 是填空式 prompt。
- prefix 是续写式 prompt。

Manual vs Automated Prompting

Manual prompt engineering:

- Human-designed templates.
- Easy but unstable and time-consuming.

Discrete prompt search:

- Search over natural-language prompt tokens.

Continuous/soft prompt:

- Learn prompt embeddings directly in vector space.
- Not necessarily human-readable.

Prefix Tuning

English: Prefix tuning prepends trainable continuous vectors to the model while freezing the original model.

中文: prefix tuning 在输入前加可训练的连续向量，原模型参数保持冻结，是参数高效调优方法之一。

Chain-of-Thought Prompting

English: CoT prompting encourages the model to produce intermediate reasoning steps before the final answer.

中文: CoT 让模型先写中间推理步骤，再给最终答案，常用于数学、逻辑、多步推理。

Types:

- Manual CoT: provide examples with reasoning.
- Zero-shot CoT: add phrase like "Let's think step by step."
- Auto-CoT: automatically construct reasoning demonstrations.
- Self-consistency: sample multiple reasoning paths and choose the majority/final consistent answer.

Alignment Methods

RLHF

- Uses reward model and reinforcement learning.
- Powerful but expensive and complex.

DPO / Direct Preference Optimization

English: DPO directly optimizes preference pairs without training a separate RL policy loop.

中文: DPO 直接利用偏好数据优化模型，相比 PPO-style RLHF 更简单、更稳定、更高效。

GRPO

English: GRPO reduces memory by avoiding a separate value network and uses group-relative rewards.

中文: GRPO 通过不使用单独 value network 等方式降低内存开销，适合大模型强化学习场景。

Lecture 9 - LLM Agents / 大模型智能体

What Is an LLM Agent?

English: An LLM agent is a system powered by an LLM that can understand goals, plan actions, use tools, interact with the environment, and complete tasks.

中文: LLM agent 是由大语言模型驱动的系统，能够理解目标、规划步骤、调用工具、根据反馈调整并完成任务。

Why Agents?

LLMs alone suffer from:

- Hallucination.
- Inaccurate/outdated knowledge.
- Limited tool-use ability.
- Lack of interpretability.
- Inconsistency.

Agents add:

- Planning.
- Tool use.
- Memory.
- Retrieval.
- Workflow control.

Agent Components

- **Planning / 规划**: decompose a goal into steps.
- **Tool use / 工具调用**: call search, calculator, database, code interpreter, APIs.
- **Memory / 记忆**: store and retrieve past interactions or facts.
- **Workflow / 工作流**: orchestrate model calls and tools.
- **Reflection/evaluation / 反思评估**: check and improve outputs.

Common Agent Workflows

- Prompt chaining.
- Orchestrator-workers.
- Evaluator-optimizer.
- Multi-agent systems.

中文:

智能体不是单次生成，而是“计划-行动-观察-修正”的循环系统。

ReAct

English: ReAct combines reasoning traces with actions: think, act, observe, repeat.

中文: ReAct 把推理和行动结合起来，模型先推理，再调用工具，根据观察结果继续推理。

Lecture 10 - Efficient Training of LLMs / 大模型高效训练

Where Does Memory Go?

For a model with (Ψ) parameters under fp16 training with Adam:

- fp16 model weights: (2Ψ) bytes.
- fp16 gradients: (2Ψ) bytes.
- Adam optimizer states:
 - fp32 parameter copy: (4Ψ)
 - fp32 momentum: (4Ψ)

- fp32 variance: $(4 \times \Psi)$

Total: $[2 \times \Psi + 2 \times \Psi + 12 \times \Psi = 16 \times \Psi]$

中文: Adam 混合精度训练中，显存不只存模型权重，还要存梯度、fp32 参数副本、momentum、variance，所以总显存约为 $(16 \times \Psi)$ bytes。

Example:

1.5B parameters:

$[16 \times 1.5B = 24GB]$

This excludes activation memory, so actual training can still OOM.

Activation Memory

Activation memory scales with:

$[\text{layers} \times \text{hidden dimension} \times \text{sequence length} \times \text{batch size}]$

中文:

激活值是前向传播中为反向传播保存的中间结果。序列越长、batch 越大、层数和隐藏维度越大，激活显存越高。

Model Parallelism

English: Split model parameters/computation across devices.

中文: 模型并行把模型不同部分分到不同 GPU 上，适合单卡放不下模型的情况。

Types:

- Tensor parallelism: split matrix operations.
- Pipeline parallelism: split layers into stages.

Data Parallelism

English: Each GPU has a full model copy but processes different mini-batch shards; gradients are synchronized.

中文: 数据并行中，每张 GPU 都有完整模型副本，处理不同数据，最后同步梯度。

Limitation:

- Basic data parallelism does not reduce per-device memory for model states.

ZeRO

English: DeepSpeed ZeRO partitions optimizer states, gradients, and parameters across data-parallel workers.

中文: ZeRO 把优化器状态、梯度、参数分片存到不同 GPU，显著降低每张卡的显存。

Stages:

- **ZeRO-1:** partition optimizer states.
- **ZeRO-2:** partition optimizer states + gradients.
- **ZeRO-3:** partition optimizer states + gradients + parameters.

Key exam point:

- DP replicates states on every GPU.
- ZeRO shards states across GPUs.

Parameter-Efficient Fine-Tuning (PEFT)

English: PEFT freezes most pretrained parameters and trains a small number of task-specific parameters.

中文: PEFT 冻结大部分预训练参数，只训练少量新增或低秩参数，以降低显存和训练成本。

Adapter

English: Adapters insert small trainable modules inside Transformer layers while freezing the base model.

中文: Adapter 在 Transformer 层中插入小型可训练模块，原模型参数冻结。

Advantage:

- Few trainable parameters.
- Good for storing multiple task-specific modules.

Limitation:

- Adds extra layers, can increase inference latency.

LoRA

English: LoRA assumes fine-tuning updates are low-rank. Instead of training full weight update (ΔW), it trains:

$$[\Delta W = BA]$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.

中文: LoRA 认为微调时权重变化可以用低秩矩阵表示，因此冻结原始权重，只训练两个小矩阵 (A,B)。

Forward:

$$[h = Wx + BAx]$$

Advantages:

- Much fewer trainable parameters.
- Lower memory.

- LoRA weights can be merged into base weights for inference, so no extra latency after merging.

Adapter vs LoRA:

- Adapter adds modules sequentially into layers.
 - LoRA modifies weight updates in parallel via low-rank matrices.
 - LoRA can be merged into original weights.
-

Lecture 11 - RAG / 检索增强生成

What Is RAG?

English: Retrieval-Augmented Generation enhances LLMs by retrieving external knowledge and grounding generation on retrieved evidence.

中文: RAG 把信息检索和文本生成结合起来，让模型先从外部知识库检索相关内容，再基于证据生成回答。

Why RAG?

- Provides up-to-date knowledge.
- Handles long-tail/domain-specific information.
- Avoids expensive retraining.
- Reduces hallucination by grounding answers.
- Enables citations and traceability.

中文:

RAG 适合知识更新快、领域资料多、不能完全依赖模型内部记忆的场景。

RAG Architecture

1. Document loading / 文档加载
2. Chunking / 文本切块
3. Embedding / 向量化
4. Indexing in vector database / 建索引
5. Query embedding / 查询向量化
6. Retrieval / 检索
7. Reranking/post-processing / 重排序与后处理
8. Generator / 生成答案

Sparse vs Dense Retrieval

Sparse retrieval

- Examples: TF-IDF, BM25.
- Based on lexical overlap.
- Good for exact keywords.
- Weak for semantic matching.

Dense retrieval

- Uses neural embeddings.
- Good for semantic similarity.
- Can retrieve relevant passages without exact word overlap.

Vector Database

English: Stores embeddings plus metadata and supports efficient nearest-neighbor search.

中文: 向量数据库存储文本块的向量和元数据，支持快速相似度检索，还可做过滤、更新、扩展。

Indexing methods:

- LSH: locality-sensitive hashing.
- PQ: product quantization.
- HNSW: graph-based approximate nearest neighbor search.

DPR and Contriever

- **DPR:** supervised dense retriever trained for QA.
- **Contriever:** unsupervised/general-purpose retriever.

Naive vs Advanced RAG

Naive RAG:

- Retrieve top-k chunks and feed them directly to LLM.

Problems:

- Irrelevant chunks.
- Missing evidence.
- Redundant chunks.
- Conflicting or stale information.

Advanced RAG:

- Query rewriting.
- Hybrid retrieval.
- Reranking.
- Metadata filtering.
- Multi-step retrieval.
- Corrective RAG.

- Citation and verification.

What/When/How to Retrieve

- **What:** which documents/chunks are useful?
 - **When:** retrieve once, iteratively, or only when needed?
 - **How:** sparse, dense, hybrid, reranked, graph-based?
-

Lecture 12 - Course Review / 总复习

Core Course Structure

Basics:

- Linguistics and preprocessing.
- Language models.
- Word embeddings.

Tasks:

- NLU: classification, QA, NER.
- NLG: MT, summarization, dialogue.

LLM topics:

- Transformer.
- Pretraining/fine-tuning.
- Prompting and alignment.
- Agents.
- Efficient fine-tuning.
- RAG.

Must-Memorize Formulas

Language model:

$$[P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{1:n-gram})]$$

$$[P(x_t | x_{1:n-gram})]$$

Perplexity:

$$[PP = P(w_1, \dots, w_T)^{-1/T}]$$

Softmax:

$$[\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}]$$

Attention:

[$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^{\text{top}}}{\sqrt{d_k}}\right)V$]

BLEU:

[$\text{BLEU}=\text{BP}\cdot \exp\left(\sum_{n=1}^N w_n \log P_n\right)$]

Precision/Recall/F1:

[$P=\frac{\text{TP}}{\text{TP}+\text{FP}}, \quad R=\frac{\text{TP}}{\text{TP}+\text{FN}}, \quad F1=\frac{2PR}{P+R}$]

LLM training memory with Adam fp16:

[$\text{Total}=16\cdot \text{Psi}$ bytes]

LoRA:

[$\Delta W=BA, \quad r \ll d$]

Common Short-Answer Templates

Explain why n-gram models suffer from sparsity.

English: The number of possible n-grams grows exponentially with vocabulary size and n, so most valid sequences are unseen. Count-based probability estimates become unreliable or zero.

中文: n-gram 的可能组合随词表大小和 n 指数增长，训练语料中大多数合法组合都没出现过，因此计数概率不可靠，甚至为 0。

Explain why Transformer is more parallelizable than RNN.

English: RNN hidden states depend sequentially on previous states, while Transformer self-attention computes relations among all positions in parallel.

中文: RNN 必须按时间步依次计算，而 Transformer 的 self-attention 可以同时计算所有位置之间的关系，因此更适合并行。

Explain why RAG reduces hallucination.

English: RAG grounds generation in retrieved external evidence, so the model can rely on source documents instead of only parametric memory.

中文: RAG 让生成结果基于检索到的外部证据，而不是只依赖模型参数记忆，因此能降低无依据编造。

Explain why LoRA is parameter-efficient.

English: LoRA freezes original weights and learns a low-rank update, so the number of trainable parameters is much smaller than full fine-tuning.

中文: LoRA 冻结原始权重，只训练低秩更新矩阵，因此可训练参数远少于全量微调。

Final Memory Checklist

- Can you define LM, n-gram, perplexity?
- Can you explain one-hot limitations and distributional hypothesis?
- Can you derive attention output from Q, K, V?

- Can you compare BERT and GPT?
- Can you compute precision/recall/F1 and BLEU?
- Can you explain greedy vs beam search?
- Can you design a task-oriented dialogue pipeline?
- Can you explain RAG architecture?
- Can you compute LLM memory ($16 \times \Psi$)?
- Can you compare Adapter and LoRA?